IMEN891K / IDSC501 / AXGS-501 · DATA ENGINEERING · FALL 2026

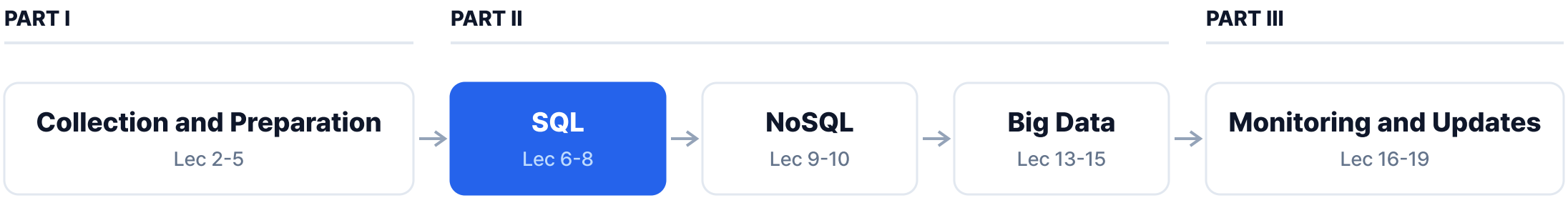
# 7.

# SQL II: Joins

Sangwoo Mo · POSTECH

Industrial & Management Engineering

# Where We Are


PART I
PART II
PART III
Collection and Preparation
Lec 2-5
SQL
Lec 6-8
NoSQL
Lec 9-10
Big Data
Lec 13-15
Monitoring and Updates
Lec 16-19

# Outline: SQL II

| | | |
|---|---|---|
| 01 | **Match Rows** | combine rows that satisfy a condition |
| 02 | **Keep Unmatched Rows** | include records with no match |
| 03 | **Test for Matches** | keep rows based on whether matches exist |
| 04 | **Stack Results** | combine rows from separate results |

# Recap: SFW on One Table

WHERE filters rows, SELECT chooses columns, and DISTINCT removes duplicate results

**`WHERE`** **selection**: keep the rows that meet a condition

**`SELECT`** **projection**: keep a subset of the columns

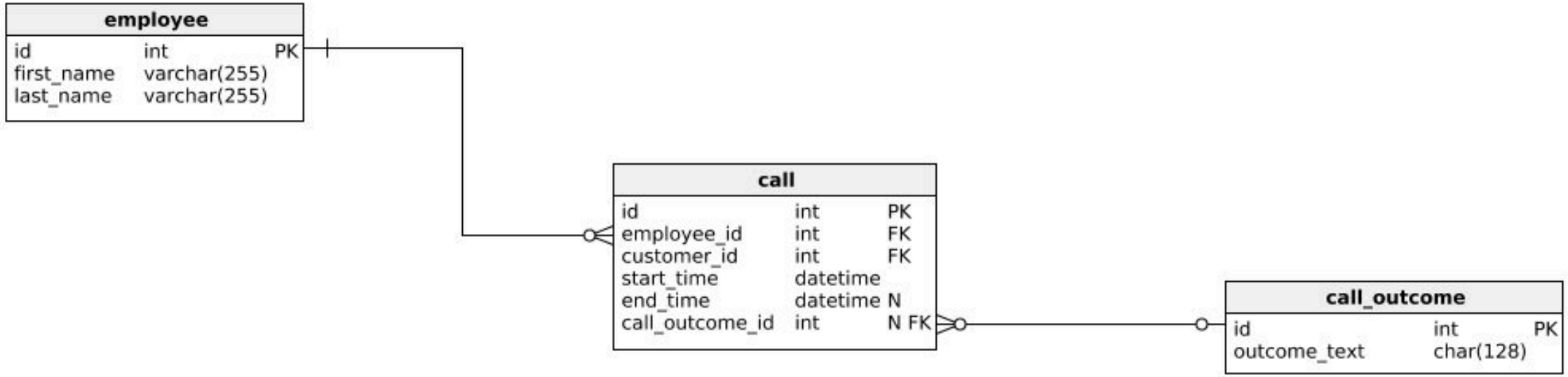
**`DISTINCT`** drop duplicate rows; a table is a **multiset**

```
SELECT DISTINCT Category
FROM   Product
WHERE  Price < 200
```

| category |
| --- |
| Gadgets |
| Photography |

Two Gadgets rows below $200 collapse into **one** with DISTINCT

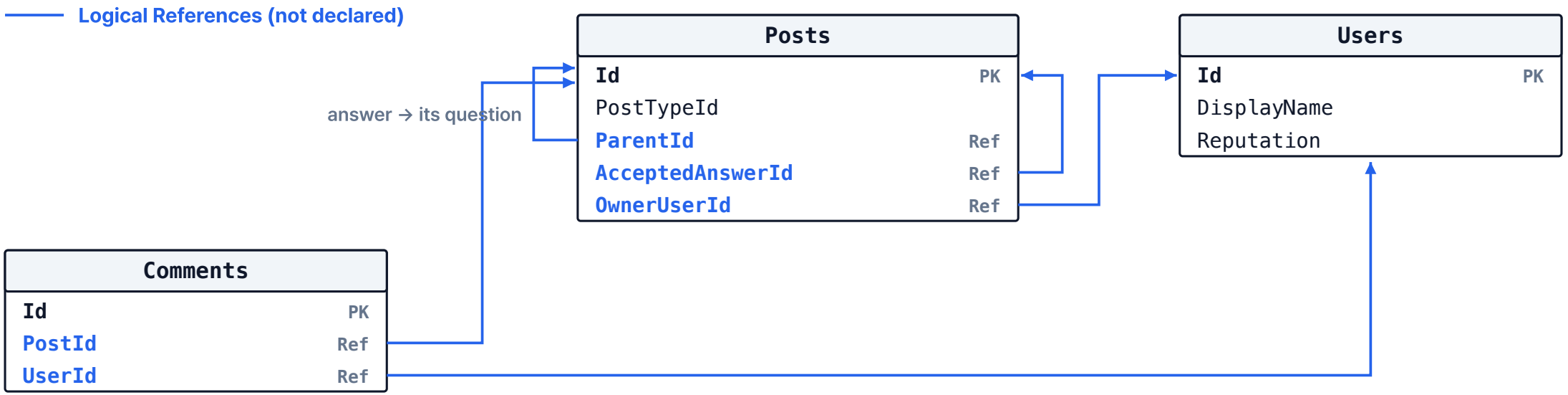
# Multi-Table Queries




Three tables linked by **foreign keys**: a question about calls per employee needs all three

# Case study: Stack Overflow

One question page is spread over several tables linked by Id columns




Answers reference questions; reconstructing a page combines **posts, users and comments**

# Recap: Product and Company

| pname | price | category | manufacturer |
|---|---|---|---|
| Gizmo | 19.99 | Gadgets | GizmoWorks |
| PowerGizmo | 29.99 | Gadgets | GizmoWorks |
| SingleTouch | 149.99 | Photography | Canon |
| MultiTouch | 203.99 | Household | Hitachi |

| cname | stockprice | country |
|---|---|---|
| GizmoWorks | 25.0 | USA |
| Canon | 65.0 | Japan |
| Hitachi | 15.0 | Japan |

Price belongs to Product; country belongs to Company; manufacturer links the two

# Table Aliases

```
SELECT P.pname,P.price,C.country
FROM Product AS P JOIN Company AS C
   ↑ P and C: short names for the tables
   ON P.manufacturer=C.cname;
```

# JOIN on Related Keys

```
SELECT P.pname,P.price,C.country
FROM Product P JOIN Company C
  ON P.manufacturer=C.cname;
```

↑ match a product to its maker

| pname | price | country |
|---|---|---|
| Gizmo | 19.99 | USA |
| PowerGizmo | 29.99 | USA |
| SingleTouch | 149.99 | Japan |
| MultiTouch | 203.99 | Japan |

Each product matches its company; the result has four product rows

# Every Pair Before the Match

```
SELECT P.pname,C.cname FROM Product P CROSS JOIN
Company C;
```

↑ every product with every company

| pname | cname |
|---|---|
| Gizmo | Canon |
| Gizmo | GizmoWorks |
| Gizmo | Hitachi |
| MultiTouch | Canon |
| MultiTouch | GizmoWorks |
| MultiTouch | Hitachi |
| PowerGizmo | Canon |
| PowerGizmo | GizmoWorks |
| PowerGizmo | Hitachi |
| SingleTouch | Canon |
| SingleTouch | GizmoWorks |
| SingleTouch | Hitachi |

Four products and three companies create twelve candidate pairs

# Predict Join Cardinality

| Relationship | Expected matching rows |
| --- | --- |
| 4 products × 3 companies | 12 candidate pairs |
| Each product has one company | 4 matching pairs |
| One company has two products | 2 rows for that company |

A join returns matching pairs; a repeated company is expected

# Filter the Joined Rows

```sql
SELECT P.pname,P.price,C.country
FROM Product P JOIN Company C ON
P.manufacturer=C.cname
WHERE C.country='Japan' AND P.price<=200;
```

↑ filter after the join

| pname | price | country |
|---|---|---|
| SingleTouch | 149.99 | Japan |

Japan and the price limit together keep SingleTouch

# Read the Query in Logical Order

| Step | Effect |
| --- | --- |
| FROM / JOIN / ON | Build matching pairs |
| WHERE | Keep the pairs that satisfy the condition |
| SELECT | Choose output columns |
| ORDER BY / LIMIT | Order and trim the result |

Logical query order explains the result without prescribing the physical execution

# Name the Output Columns

```
SELECT P.pname AS product,C.country AS
manufacturer_country
    ↑ name of the output column
FROM Product P JOIN Company C ON
P.manufacturer=C.cname;
```

| product | manufacturer_country |
|---|---|
| Gizmo | USA |
| PowerGizmo | USA |
| SingleTouch | Japan |
| MultiTouch | Japan |

Output aliases describe what each selected column means

# Caveat: Rows Multiply at a Join

```
SELECT C.cname,P.pname
FROM Company C JOIN Product P ON
P.manufacturer=C.cname;
```

↑ one row per matching product

| cname | pname |
|---|---|
| GizmoWorks | Gizmo |
| GizmoWorks | PowerGizmo |
| Canon | SingleTouch |
| Hitachi | MultiTouch |

GizmoWorks appears twice because it matches two different products

# Self Join: Compare Products

```
SELECT A.pname AS first_product,B.pname
AS second_product
FROM Product A JOIN Product B
   ↑ one table, two names
   ON A.manufacturer=B.manufacturer
WHERE A.pname<>B.pname;
   ↑ not paired with itself
```

| first_product | second_product |
|---|---|
| Gizmo | PowerGizmo |
| PowerGizmo | Gizmo |

The same two products appear in both comparison orders

# Keep Each Product Pair Once

```
SELECT A.pname AS first_product,B.pname
AS second_product
FROM Product A JOIN Product B
  ON A.manufacturer=B.manufacturer
WHERE A.pname<B.pname;
```

↑ < keeps each pair once

| first_product | second_product |
| --- | --- |
| Gizmo | PowerGizmo |

An ordered key comparison keeps the pair only once

# Recap: The Join Input

| pname | price | manufacturer |
|---|---|---|
| Gizmo | 19.99 | GizmoWorks |
| PowerGizmo | 29.99 | GizmoWorks |
| SingleTouch | 149.99 | Canon |
| MultiTouch | 203.99 | Hitachi |

| cname | country |
|---|---|
| GizmoWorks | USA |
| Canon | Japan |
| Hitachi | Japan |

Two companies are in Japan; the price condition decides which products remain

# Demo: Match Products to Companies

1. Return names and prices of products made in Japan and priced at most 200
2. Remove the price limit and compare the rows

```
-- write your query here, then Run
```

**QUERY RESULT**

Write a query on the left, then Run.

# Answer: Japanese Products

```
SELECT P.pname,P.price FROM Product P
JOIN Company C ON P.manufacturer=C.cname
WHERE C.country='Japan';
```

| pname | price |
| --- | --- |
| SingleTouch | 149.99 |
| MultiTouch | 203.99 |

Without the price limit, both SingleTouch and MultiTouch remain

# A Company Without Products

| cname | stockprice | country |
|---|---|---|
| GizmoWorks | 25.0 | USA |
| Canon | 65.0 | Japan |
| Hitachi | 15.0 | Japan |
| NewCo | 0.0 | Korea |

| pname | manufacturer |
|---|---|
| Gizmo | GizmoWorks |
| PowerGizmo | GizmoWorks |
| SingleTouch | Canon |
| MultiTouch | Hitachi |

NewCo is a company with no matching product in this copy

# INNER JOIN and LEFT JOIN

```
SELECT C.cname,P.pname FROM Company C
JOIN Product P ON P.manufacturer=C.cname;
```

| cname | pname |
|---|---|
| GizmoWorks | Gizmo |
| GizmoWorks | PowerGizmo |
| Canon | SingleTouch |
| Hitachi | MultiTouch |

```
SELECT C.cname,P.pname FROM Company C
LEFT JOIN Product P ON P.manufacturer=C.cname;
```

↑ companies with no product stay, with NULL

| cname | pname |
|---|---|
| Canon | SingleTouch |
| GizmoWorks | Gizmo |
| GizmoWorks | PowerGizmo |
| Hitachi | MultiTouch |
| NewCo | NULL |

LEFT JOIN adds NewCo with a NULL product; INNER JOIN has no row for it

# Find an Unmatched Company

```
SELECT C.cname FROM Company C
LEFT JOIN Product P ON P.manufacturer=C.cname
WHERE P.pname IS NULL;
```

↑ no match: the product side is NULL

| cname |
| --- |
| NewCo |

The right-side product key is NULL only for NewCo

# Start from the Rows You Must Keep

```
SELECT C.cname,P.pname FROM Company C
LEFT JOIN Product P ON P.manufacturer=C.cname
ORDER BY C.cname,P.pname;
```

| cname | pname |
| --- | --- |
| Canon | SingleTouch |
| GizmoWorks | Gizmo |
| GizmoWorks | PowerGizmo |
| Hitachi | MultiTouch |
| NewCo | NULL |

The left input determines which unmatched rows survive

# Caveat: ON and WHERE

```
SELECT C.cname,P.pname FROM Company C
LEFT JOIN Product P ON P.manufacturer=C.cname
AND P.price>100;
```

↑ in ON: every company is kept

| cname | pname |
|---|---|
| Canon | SingleTouch |
| GizmoWorks | NULL |
| Hitachi | MultiTouch |
| NewCo | NULL |

```
SELECT C.cname,P.pname FROM Company C
LEFT JOIN Product P ON P.manufacturer=C.cname
WHERE P.price>100;
```

↑ in WHERE: NULL rows are dropped

| cname | pname |
|---|---|
| Canon | SingleTouch |
| Hitachi | MultiTouch |

ON limits matches while keeping companies; WHERE removes rows after the join

# Recap: The Unmatched Company

| cname | country |
| --- | --- |
| GizmoWorks | USA |
| Canon | Japan |
| Hitachi | Japan |
| NewCo | Korea |

Keep all four companies, including NewCo with no product match

# Demo: Keep Companies Without Products

1. List every company with its product names, including companies with no products
2. Keep only companies with no matching product

```
-- write your query here, then Run
```

**QUERY RESULT**

Write a query on the left, then Run.

# Answer: Companies Without Products

```
SELECT C.cname FROM Company C
LEFT JOIN Product P ON P.manufacturer=C.cname
WHERE P.pname IS NULL;
```

| cname |
| --- |
| NewCo |

NewCo exists in Company but has no matching product

# Naming a Query Result

```
WITH japanese_products AS (
   ↑ a name for this result
   SELECT P.pname,P.price FROM Product P
   JOIN Company C ON P.manufacturer=C.cname
   WHERE C.country='Japan'
)
SELECT * FROM japanese_products;
```

| pname | price |
|---|---|
| SingleTouch | 149.99 |
| MultiTouch | 203.99 |

WITH gives a query result a name that another query can read

# A Derived Table Is Another Input

```
SELECT G.pname,G.price FROM
  (SELECT pname,price FROM Product
WHERE category='Gadgets') AS G
     ↑ the subquery is used as a table
WHERE G.price>20;
```

| pname | price |
|---|---|
| PowerGizmo | 29.99 |

A query result can be read as a table by another query

# EXISTS and Query Blocks

```
SELECT C.cname FROM Company C
WHERE EXISTS (
  SELECT 1 FROM Product P
  WHERE P.manufacturer=C.cname AND P.category='Gadgets'
);
```

true if the inner query returns a row

# IN Reads a Query Result

```
SELECT cname FROM Company
WHERE cname IN (SELECT manufacturer FROM Product
  ↑ the list comes from a query
  WHERE category='Gadgets');
```

| cname |
| --- |
| GizmoWorks |

The inner query supplies candidate manufacturer names

# Inspect the Inner Result First

```sql
SELECT manufacturer FROM Product WHERE
category='Gadgets';
```

| manufacturer |
| --- |
| GizmoWorks |
| GizmoWorks |

Repeated candidate values do not repeat the outer company

# JOIN or EXISTS

```
SELECT C.cname FROM Company C JOIN Product P
ON P.manufacturer=C.cname WHERE
P.category='Gadgets';
```

| cname |
| --- |
| GizmoWorks |
| GizmoWorks |

```
SELECT C.cname FROM Company C WHERE EXISTS (
  SELECT 1 FROM Product P
  WHERE P.manufacturer=C.cname AND
P.category='Gadgets');
```

| cname |
| --- |
| GizmoWorks |

JOIN returns two Gadget matches; EXISTS keeps one company row

# NOT EXISTS

```
SELECT C.cname FROM Company C WHERE NOT EXISTS (
  ↑ keep companies with no such product
  SELECT 1 FROM Product P
  WHERE P.manufacturer=C.cname AND
P.category='Gadgets');
```

| cname |
| --- |
| Canon |
| Hitachi |
| NewCo |

NOT EXISTS keeps Canon, Hitachi and NewCo, which have no Gadget match

# Recap: Company-Level Questions

| cname |
| --- |
| Canon |
| GizmoWorks |
| Hitachi |
| NewCo |

Each answer should contain company rows, without repeating a company per product

# Demo: Ask Whether a Match Exists

1. Find companies that make a Gadget with `EXISTS`
2. Find companies that make no Gadgets with `NOT EXISTS`

```
-- write your query here, then Run
```

**QUERY RESULT**

Write a query on the left, then Run.

# Answer: Match Existence

```
SELECT C.cname FROM Company C WHERE NOT EXISTS (
  SELECT 1 FROM Product P
  WHERE P.manufacturer=C.cname AND
P.category='Gadgets');
```

| cname |
| --- |
| Canon |
| Hitachi |
| NewCo |

Three companies have no Gadget; the matching company is GizmoWorks

# Matching and Stacking Answer Different Questions

| Question | Operation |
| --- | --- |
| Which rows belong together? | JOIN |
| Does a related row exist? | EXISTS |
| Which rows should survive without a match? | LEFT JOIN / NOT EXISTS |
| Which compatible lists should be combined? | UNION ALL / UNION |

The question determines whether to match pairs, test existence or stack rows

# UNION ALL and UNION

```
SELECT pname FROM Product WHERE category='Gadgets'
UNION ALL
SELECT pname FROM Product WHERE price<200;
```

stack both results, duplicates kept

UNION ALL stacks compatible result rows; UNION also removes duplicate rows

# INTERSECT Keeps Shared Values

```
SELECT cname FROM Company
INTERSECT SELECT manufacturer FROM Product;
```

↑ values in both results

| cname |
| --- |
| Canon |
| GizmoWorks |
| Hitachi |

INTERSECT retains company names present in both results

# EXCEPT Keeps Values Only on the Left

```
SELECT cname FROM Company
EXCEPT SELECT manufacturer FROM Product;
```

↑ left result minus right result

| cname |
| --- |
| NewCo |

EXCEPT finds NewCo because it occurs only in the company list

# Compatible Result Columns

| First query | Second query | Combination |
| --- | --- | --- |
| pname | pname | One name column |
| pname, price | pname, price | Two corresponding columns |
| pname, price | pname | Invalid: different column counts |

Stack corresponding columns in the same order

# Recap: Overlapping Product Lists

| pname | price | category |
|---|---|---|
| Gizmo | 19.99 | Gadgets |
| PowerGizmo | 29.99 | Gadgets |
| SingleTouch | 149.99 | Photography |
| MultiTouch | 203.99 | Household |

The Gadget list overlaps the below-200 list at Gizmo and PowerGizmo

# Demo: Stack Two Product Lists

1. Combine Gadget names and below-200 product names with `UNION ALL`
2. Change it to `UNION` and explain the difference

```
-- write your query here, then Run
```

**QUERY RESULT**

Write a query on the left, then Run.

# Answer: Stack Product Lists

```
SELECT pname FROM Product WHERE category='Gadgets'
UNION
SELECT pname FROM Product WHERE price<200;
```

| pname |
| --- |
| Gizmo |
| PowerGizmo |
| SingleTouch |

UNION keeps three distinct names; UNION ALL keeps all five input rows

# Filter the Whole Combined Result

```
SELECT manufacturer FROM (
  SELECT manufacturer FROM Product WHERE
category='Gadgets'
  UNION ALL
  SELECT manufacturer FROM Product WHERE price>20
) AS combined WHERE manufacturer='GizmoWorks';
```

↑ filter after combining

| manufacturer |
| --- |
| GizmoWorks |
| GizmoWorks |
| GizmoWorks |

The outer condition applies to both stacked inputs

# Takeaway: SQL II

1. **JOIN and LEFT JOIN** control matching pairs and unmatched rows
2. **EXISTS and NOT EXISTS** answer whether a related row exists
3. **UNION ALL and UNION** stack compatible results with an explicit duplicate policy